

100 JavaScript Interview Questions & Answers (Detailed with Examples)

This document contains beginner to advanced JavaScript interview questions with clear explanations and examples. Useful for interviews, revision, and real-world understanding.

1. What is JavaScript?

JavaScript is a high-level, interpreted programming language used to create dynamic and interactive web pages. It runs in the browser and on servers using Node.js.

2. Difference between var, let, and const?

var is function-scoped and can be redeclared. let and const are block-scoped; let allows reassignment, const does not.

3. What are arrow functions?

Arrow functions provide a shorter syntax and do not have their own this. Example: `const add = (a,b) => a+b.`

4. What is a closure?

A closure is a function that remembers variables from its outer scope even after the outer function has executed.

5. Difference between == and ===?

== compares values with type coercion, === compares both value and type without coercion.

6. What is the DOM?

The DOM is a tree-like representation of HTML that allows JavaScript to manipulate web page content dynamically.

7. What is hoisting?

Hoisting moves variable and function declarations to the top of their scope during execution.

8. What are template literals?

Template literals use backticks and allow interpolation: ``Hello ${name}`.`

9. Difference between undefined and null?

undefined means a variable is declared but not assigned; null is an intentional empty value.

10. How does setTimeout work?

setTimeout executes a function after a delay using the event loop and task queue.

11. What are higher-order functions?

Functions that take other functions as arguments or return functions, e.g., map, filter.

12. bind(), call(), apply()?

They control the value of this. call and apply invoke immediately; bind returns a new function.

13. Event bubbling and capturing?

Bubbling moves events from child to parent; capturing moves from parent to child.

14. Spread operator?

Expands arrays or objects: [...arr], {...obj}.

15. Rest operator?

Collects multiple arguments into an array using ...rest.

16. Callback functions?

Functions passed as arguments and executed later.

17. Synchronous vs Asynchronous?

Synchronous blocks execution; asynchronous is non-blocking.

18. What is a Promise?

A Promise represents a future value: pending, fulfilled, or rejected.

19. async/await?

Syntactic sugar over promises to write async code like sync code.

20. map vs forEach vs filter?

map returns transformed array, forEach loops, filter returns matching elements.

21. What is IIFE?

A function that runs immediately after creation.

22. Lexical scoping?

Scope determined by code structure, not runtime.

23. this keyword?

Refers to the object calling the function.

24. Stack vs Heap?

Stack stores primitives, heap stores objects.

25. How to clone an object?

Use spread for shallow copy, JSON methods for deep copy.

26. Event delegation?

Using a single parent event listener for multiple children.

27. Function declaration vs expression?

Declarations are hoisted, expressions are not.

28. Pure function?

A function with no side effects and same output for same input.

29. slice vs splice?

slice does not modify array; splice modifies it.

30. Event loop?

Manages call stack, microtask queue, and task queue.

31. Promise chain?

Multiple .then() calls passing results sequentially.

32. Optional chaining?

Safely access nested properties using ?.

33. Nullish coalescing?

?? returns right value only if left is null or undefined.

34. Sync vs Async?

Sync blocks execution; async allows parallel operations.

35. Thunk?

A function that delays execution.

36. for...of vs for...in?

for...of iterates values, for...in iterates keys.

37. Memoization?

Caching results to improve performance.

38. Debouncing?

Executes function after events stop firing.

39. Throttling?

Limits function execution to fixed intervals.

40. Data attributes?

Custom HTML attributes using data-.*.

41. Types of errors?

Syntax, Reference, Type, Range, Logical errors.

42. Error handling?

Using try, catch, finally.

43. this in JS?

Context-dependent reference to calling object.

44. How bind works?

Returns a new function with fixed this.

45. call vs apply?

Difference is argument passing style.

46. Generator function?

Functions that pause using yield.

47. new keyword?

Creates object, sets prototype and this.

48. Template engines?

Generate dynamic HTML (EJS, Handlebars).

49. innerHTML vs textContent?

innerHTML parses HTML; textContent is safe text.

50. Strict mode?

Enforces safer JavaScript with 'use strict'.

51. Object.freeze()?

Prevents object modification.

52. Symbol?

Unique primitive used as object keys.

53. Rest parameters?

Collect multiple arguments into array.

54. Clone object?

Spread or JSON methods.

55. Deep vs Shallow copy?

Deep copies nested objects; shallow does not.

56. typeof null?

Returns 'object' due to JS bug.

57. Falsy values?

false, 0, "", null, undefined, NaN.

58. Check array?

Use Array.isArray().

59. JSON.parse/stringify?

Convert between object and JSON string.

60. Tagged template literal?

Function that processes template literals.

61. JavaScript modules?

Allow splitting code using import/export.

62. WeakMap?

Key-value store with weak object keys.

63. Currying?

Transform function into nested single-arg functions.

64. Default parameters?

Provide fallback values for function params.

65. Compare objects?

Objects compared by reference, not value.

66. Prototype chaining?

Inheritance mechanism in JavaScript.

67. == vs Object.is()?

Object.is handles NaN and -0 correctly.

68. Call stack?

Tracks function execution order.

69. Microtask queue?

Handles promise callbacks.

70. Destructuring?

Extract values from arrays/objects.

71. Optional parameters?

Parameters that may or may not be passed.

72. Role of use strict?

Prevents unsafe actions and silent errors.

73. String to number?

Use Number(), parseInt(), parseFloat().

74. Remove duplicates from array?

Use Set or filter.

75. reduce()?

Reduces array to single value.

76. instanceof?

Checks prototype chain.

77. Promise.all()?

Runs multiple promises in parallel.

78. Ternary operator?

Short if-else: condition ? a : b.

79. Destructuring with aliasing?

Rename variables during destructuring.

80. Flatten array?

Use flat() or recursion.

81. Dynamic typing?

Types determined at runtime.

82. Factory function?

Function returning new objects.

83. Lexical scope?

Scope based on code structure.

84. Polyfill?

Adds missing features for older browsers.

85. seal vs freeze?

seal allows modification; freeze does not.

86. Global variables?

Declared outside functions.

87. Prevent object modification?

freeze, seal, preventExtensions.

88. Callback hell?

Nested callbacks causing unreadable code.

89. Higher-order component?

Function that returns a component.

90. Random numbers?

Use Math.random().

91. NaN?

Invalid number result.

92. Template literals?

String interpolation using backticks.

93. Merge arrays?

Use spread or concat.

94. Constructor function?

Creates objects with new.

95. Default case in switch?

Runs when no case matches.

96. Recursion?

Function calling itself.

97. Compare arrays?

Compare values, not references.

98. bind()?

Returns new function with fixed this.

99. WeakSet?

Stores weak object references.

100. How modules work?

Encapsulated code using import/export.